

active-slam-rl

The File-by-File Guide

Every directory, every file, what's actually inside it, and why it needs to exist — a companion to `active_slam_rl_explained.pdf`, organized by repository structure instead of concept flow.

How this document differs from [active_slam_rl_explained.pdf](#): that document walks through the algorithm in the order data flows through it (sensing → registration → mapping → decision → action). This one walks through the repository the way you'd explore it in a file browser — directory by directory, file by file — so you can look up “what is this file and why does it exist” for anything you're looking at. The same diagrams appear in both, attached to whichever file actually implements that piece of math.

Every file gets: what's literally defined inside it (the classes/functions), and a boxed “**Why this file exists**” note explaining the conceptual reason it's a separate piece rather than folded into something else.

/ (repository root)

Dockerfile, docker-compose.yml, .dockerignore

A multi-stage-free Dockerfile: Python 3.11-slim base, installs build-essential/cmake (for optionally compiling the native FS2D library later), installs requirements.txt, then installs this package itself in editable mode. docker-compose.yml wraps `docker run` with the volume mount that makes results/ persist on your host machine.

Why this file exists: Reproducibility. “It works on my machine” is a real problem in research code with a deep dependency chain (PyTorch, Stable-Baselines3, specific NumPy/SciPy versions) — the Dockerfile makes that chain a one-time, one-command build for anyone, including future-you on a different machine.

pyproject.toml

A minimal setuptools config: package name, version, and `where = ["src"]` telling Python where the actual package lives.

Why this file exists: Lets you run `pip install -e .` once and then `import active\_slam\_rl` from anywhere — no PYTHONPATH hacks, no relative-import fragility.

requirements.txt

Pinned minimum versions for torch, gymnasium, stable-baselines3, numpy, scipy, matplotlib, pandas, pyyaml, pytest, pillow.

Why this file exists: One file, one command (`pip install -r requirements.txt`), and both a human and the Dockerfile install the exact same dependency set.

.gitignore

Excludes `\_\_pycache\_\_`, `.venv`, build artifacts, and most of `results/` (trained models, logs, generated plots/GIFs) from version control.

Why this file exists: Keeps the git history to source code and configuration — large binary training artifacts belong in the working directory, not in every future git clone.

configs/

Every hyperparameter used anywhere in the project lives in exactly one of these two YAML files — nothing is hard-coded inside the Python modules themselves.

`default.yaml`

The full-scale configuration: a 140×140 world, 900-step corridor generation, PPO with `n_envs=4` and 200,000 timesteps. Meant for a real, long training run.

`quick_demo.yaml`

A deliberately small, fast configuration: 90×90 world, smaller sonar frame, single environment, ~18,000 timesteps. This is the exact configuration that produced every real result in `active_slam_rl_explained.pdf`, and it's what you should use while iterating on changes.

Why this file exists: Separating these two means you can smoke-test a change in minutes on ``quick_demo.yaml`` before committing to a multi-hour run on ``default.yaml``. Critically: a saved model's network shape is baked in by its config's world/sonar/patch dimensions, so a model trained with one config cannot be loaded against the other — that's why every script takes an explicit ``--config`` argument instead of assuming one.

native/fs2d/

Constructor University's real FS2D registration library is compiled C/C++ that this project doesn't have access to. This directory is the documented doorway for dropping it in later without touching a single line of Python elsewhere in the repo.

fs2d_c_api.h

A C header declaring exactly one function: ``fs2d_register(scan_prev, scan_curr, rows, cols, out_dx, out_dy, out_dtheta, out_quality, out_cov9)``. Flat C arrays in, flat C arrays out — no C++ classes crossing the boundary.

fs2d_c_api_template.cpp

A stub implementation of that header matching the exact signature, with comments showing where to call into the real FS2D internals and fill the output pointers from its result.

CMakeLists.txt

Builds ``fs2d_c_api_template.cpp`` (or, once renamed, the real implementation) into ``libfs2d.so`/`.dylib`/`.dll`` under ``native/fs2d/build``.

README.md

The exact build steps, and how ``registration/fs2d.py``'s ``NativeFS2DBinding`` finds and loads the resulting shared library automatically via ``ctypes``.

Why this file exists: This lets development and training happen today against a faithful NumPy reference implementation, with a clearly documented, mechanical (not a redesign) path to swap in the real compiled library later — every other module only ever sees a ``RegistrationResult`` object, never which backend produced it.

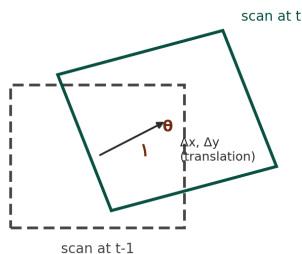
src/active_slam_rl/registration/

fs2d.py

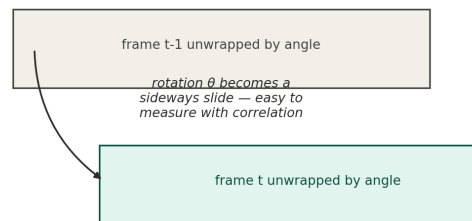
Defines: `RegistrationResult` (the dataclass every downstream module consumes — `dx`, `dy`, `dtheta`, `quality`, `covariance`, `backend`); `FourierMellinRegistration` (the real, working NumPy/SciPy implementation — log-polar phase correlation for rotation, ordinary phase correlation for translation); `NativeFS2DBinding` (the ctypes FFI wrapper for the real library, see `native/fs2d/`); `FS2DRegistration` (the facade that tries native first, falls back to NumPy automatically).

FS2D: finding motion between two noisy sonar frames

Two misaligned sonar frames

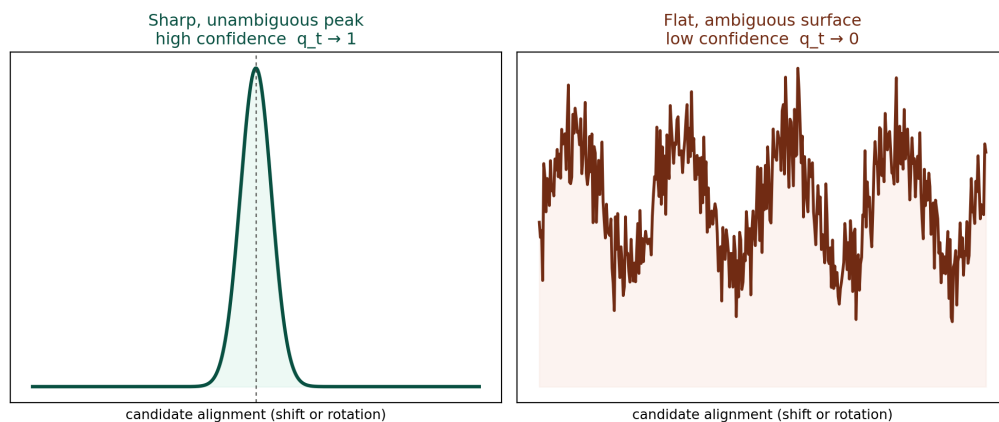


Log-polar re-plot: rotation = shift

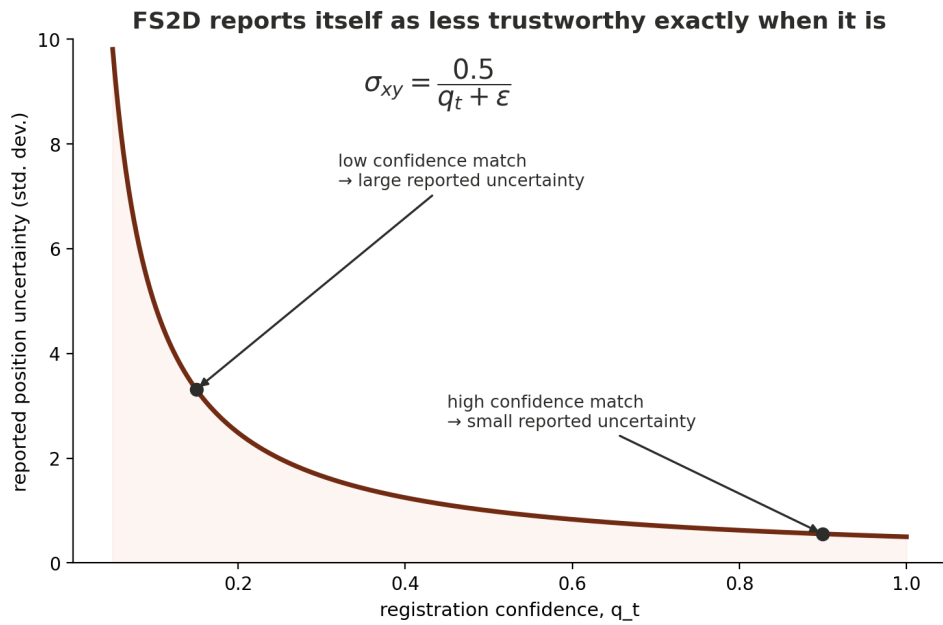


How rotation becomes a measurable shift once re-plotted in log-polar coordinates.

Registration confidence = how sharp the correlation peak is



Confidence (q_t) is read directly from how sharp the correlation peak is.



The covariance handed downstream grows exactly as confidence drops -- the literal $\sigma_{xy} = 0.5 / (q_t + \epsilon)$ relationship.

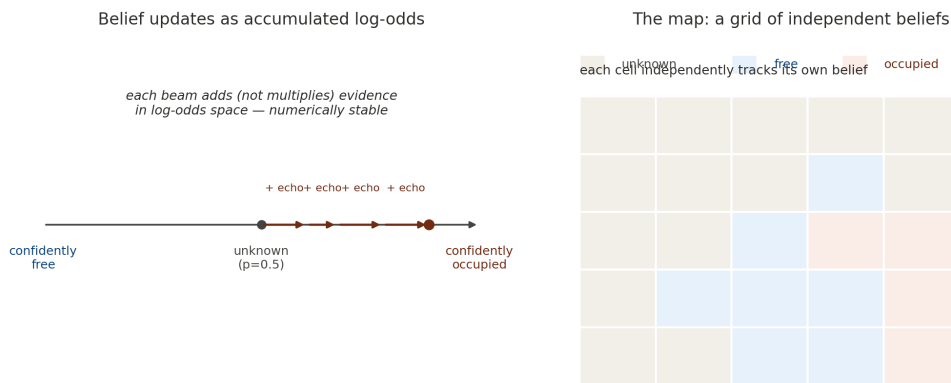
Why this file exists: Sonar frames are grainy (speckle noise) in a way that breaks ordinary pixel-matching, but survives much better in the Fourier domain. This file is isolated from the mapping/perception code specifically so the native C++ swap (native/fs2d/) only ever has to satisfy this one file's contract.

src/active_slam_rl/mapping/

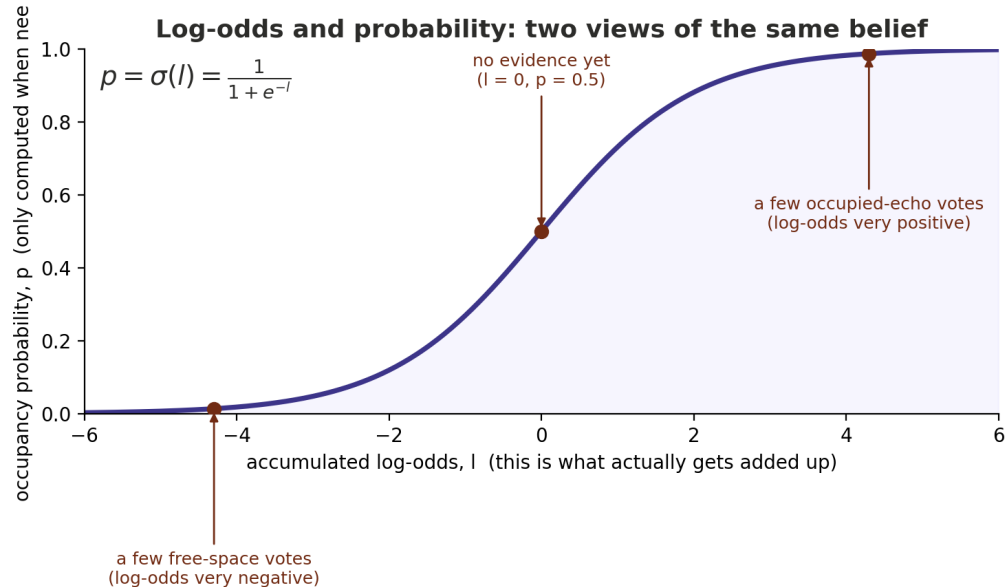
volumetric_map.py

Defines `OccupancyGrid`: a log-odds occupancy grid with `update_beam()` (Bayesian evidence accumulation along one sonar beam), `.prob` (current belief, converted from log-odds on demand), `.uncertainty()` (Bernoulli variance per voxel), `.entropy()`/`.entropy_normalized()` (the reward's coverage signal), and `.completeness()` (the section-8 evaluation metric).

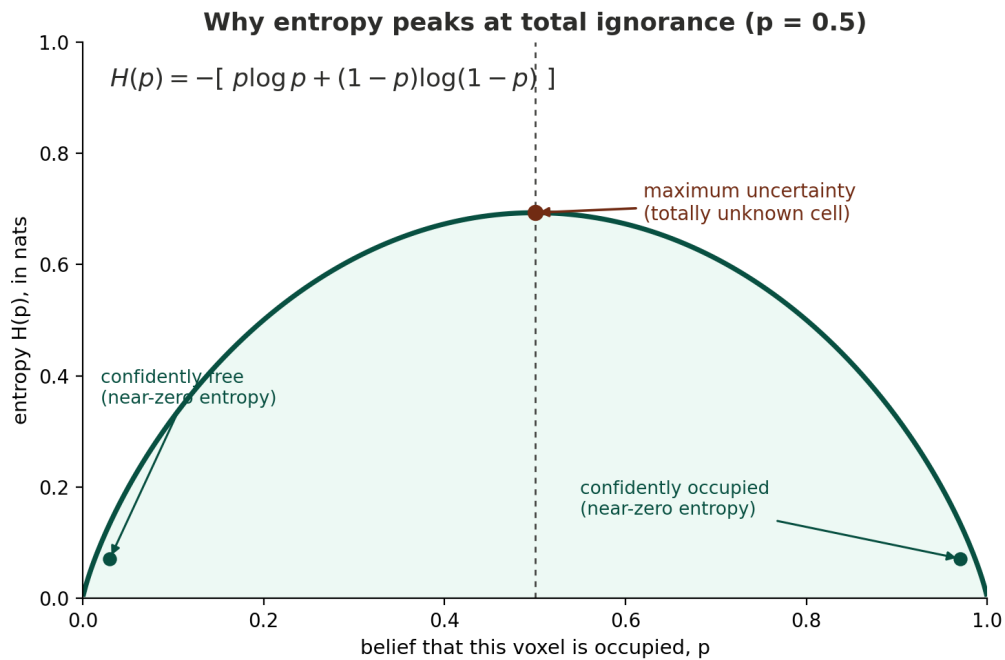
Bayesian occupancy mapping



Evidence accumulates additively in log-odds space; the map is a grid of independent beliefs.



Log-odds (unbounded, safe to add) and probability (bounded, easy to interpret) -- the sigmoid between them.



Entropy peaks exactly at total ignorance ($p=0.5$) -- this is what the reward's coverage term actually reduces.

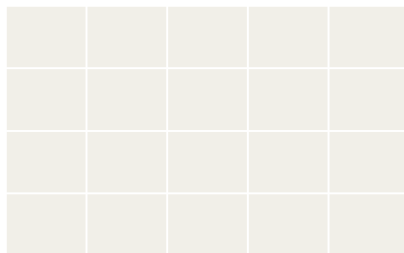
Why this file exists: A cell's belief needs to be built from many noisy observations over time without becoming numerically unstable (raw probability multiplication underflows fast) or overconfident from a single bad reading (hence the log-odds clamp ``l_max``). Isolating this in one class means the reward, the state encoder, and the completeness metric can all just read ``prob`/`.uncertainty()`` without knowing anything about log-odds internals.

change_detection.py

Defines ``compute_innovation()`` (the normalized-surprise formula), ``compute_change_mask()`` (thresholds it into a boolean mask), and ``change_cluster_stats()`` (groups flagged voxels into connected regions for the reward/state to act on as discrete targets).

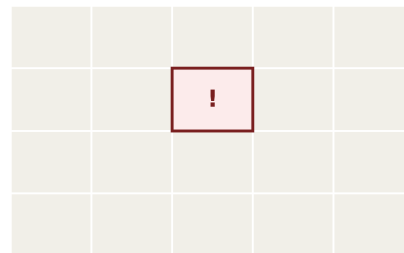
Change detection: is this a real surprise, or just noise?

Belief at $t-1$



quiet, mostly-resolved map

Belief at t (after a new scan)



big jump in a cell that seemed settled → flagged as real change

The same size belief-jump is unremarkable in an uncertain cell, but a real surprise in a settled one.

Change detection: is this jump bigger than expected noise?

denominator: how much jumping
around was already expected here

$$\eta_t^v = \frac{|p_t(v) - p_{t-1}(v)|}{\sqrt{\text{Var}[p_t(v)] + \varepsilon}}$$

numerator: how much
the belief just jumped

big →

flag as real change

The literal formula: jump size divided by how much jumping was already expected here.

Why this file exists: This is a pure-function module (no state of its own) deliberately kept separate from `OccupancyGrid` — it only ever reads two belief snapshots and a variance field, so it can be tested and reasoned about in complete isolation from how those snapshots were produced.

src/active_slam_rl/perception/

loop_closure.py

Defines `SonarDescriptor` (a rotation-invariant radial-intensity-histogram fingerprint of a sonar frame), `Keyframe`/`LoopClosureCandidate` (small dataclasses), and `LoopClosureDetector` (maintains the keyframe database, `maybe_add_keyframe()` decides when a new one is worth storing, `query()` returns the saliency score and best match).

Loop closure: recognizing a place corrects accumulated drift

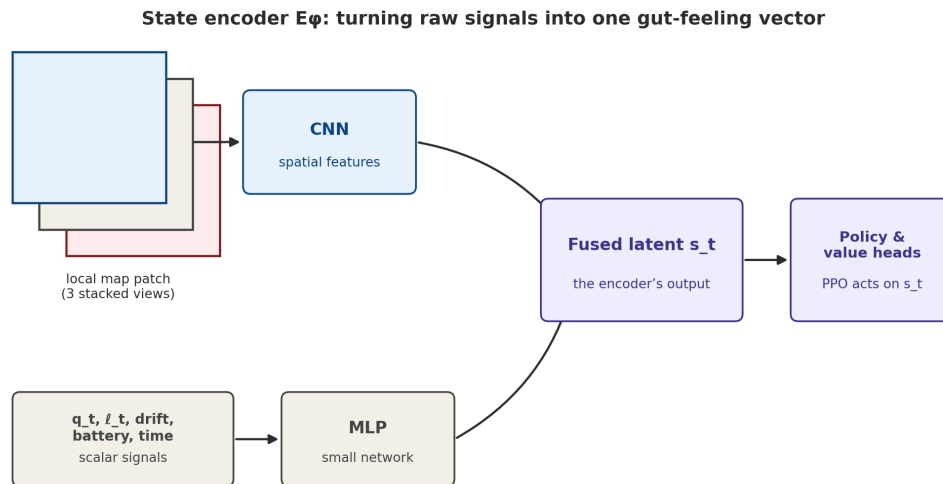


Dead-reckoning drift, corrected the moment a familiar place is recognized.

Why this file exists: Dead reckoning always drifts — there's no GPS underwater to correct it externally. This module is the only source of an external correction signal in the whole pipeline, which is why it's given its own directory rather than being folded into mapping: it's conceptually a different job (recognizing **where you've been**, not building **what things look like**).

encoder.py

Defines `PatchCNN` (small conv stack over the 3-channel local map crop), `ScalarMLP` (fuses registration quality, loop-closure saliency, drift, battery/time), `StateEncoder` (concatenates and projects both into the latent s_t), and `SB3StateEncoderExtractor` (the adapter letting Stable-Baselines3 use `StateEncoder` as PPO's features extractor).



trained jointly with the policy — gradients decide what the encoder pays attention to

A small CNN reads the map crop; a small MLP reads the scalar signals; both fuse into one vector.

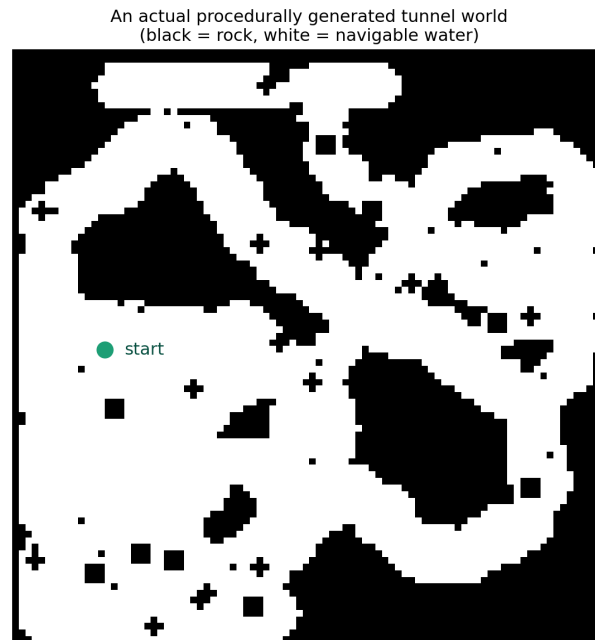
Why this file exists: The policy can't be shown the raw map or sonar frame directly — too large, too variable-sized. This file exists specifically so the *same* network that will be trained end to end with the policy (feedback loop G: policy gradients reshape what the encoder pays attention to) is defined once, in a form Stable-Baselines3's API can consume directly.

src/active_slam_rl/env/

This directory is the simulated world the rest of the pipeline runs inside, plus the reward function and the two real-simulator adapters.

world_generator.py

Defines `WorldConfig` and `TunnelWorld`: a procedural tunnel/corridor generator using a randomized-width random walk, with optional branches, scattered debris, and an optional loop-back so a genuine loop-closure opportunity exists.



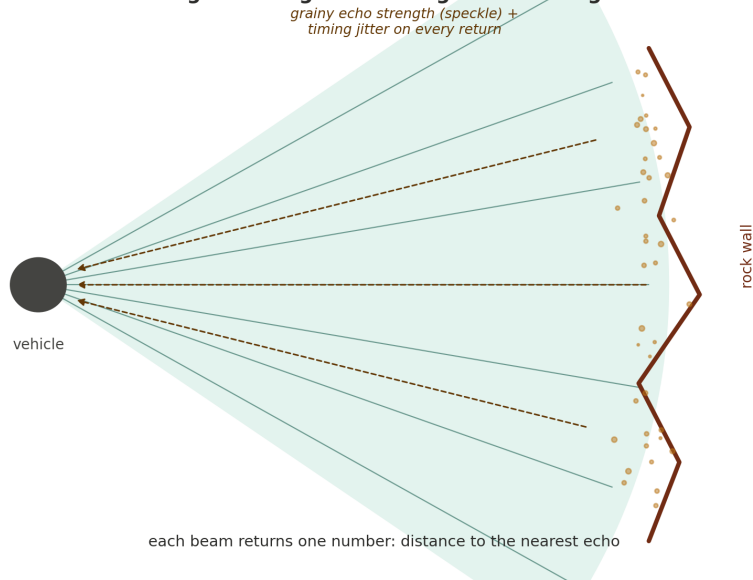
An actual generated tunnel world (not a mockup) -- black is rock, white is navigable water.

Why this file exists: Every training episode needs a genuinely different layout, or the policy just memorizes one map instead of learning to explore. Procedural generation with a seed gives unlimited, reproducible variety at zero authoring cost.

sonar_model.py

Defines `SonarConfig` and `SonarModel`: beam raycasting against `TunnelWorld.is_free()`, with `sense_imaging()` (wide-FOV imaging sonar) and `sense_scanning_360()` (narrow-beam full-sweep scanning sonar), both returning `(ranges, angles, hit_points, frame)`.

Sonar sensing: shouting into the fog and listening for echoes



Each beam returns one number: distance to the nearest echo, with realistic noise.

Why this file exists: Isolating sensing from the world means the exact same raycasting logic works whether the underlying world is `TunnelWorld` or, later, a MarineGym/Stonefish scene — only the collision query (`is\_free`) needs to change.

reward.py

Defines `RewardWeights` (five tunable weights) and `compute\_reward()` (a pure function: entropy_delta, info_gain, proximity_cost, loop_closure_validated, change_voxels_resolved in; a `RewardBreakdown` out).

The reward: five physically-motivated terms added together

Map entropy reduced	reward genuine new knowledge, not just movement
Loop-closure info gain	reward correcting real accumulated drift
Proximity / collision	penalize getting close to or hitting a wall
Loop-closure bonus	flat reward for a validated, confident recognition
Change-region resolved	reward going back to check a flagged surprise

Five physically-motivated terms, added together.

The reward equation, term by term

$$r_t = \underbrace{w_{cov} \Delta H_t}_{\text{map entropy reduced}} + \underbrace{w_{cons} \Delta I_t}_{\text{loop-closure info gain}} - \underbrace{w_{safe} C_{prox, t}}_{\text{proximity / collision}} + \underbrace{w_{loop} b_{lc, t}}_{\text{loop-closure bonus}} + \underbrace{w_{chg}(\dots)}_{\text{change region resolved}}$$

each weight (w_{cov} , w_{cons} , w_{safe} , w_{loop} , w_{chg}) is a knob in `RewardWeights` — `env/reward.py`

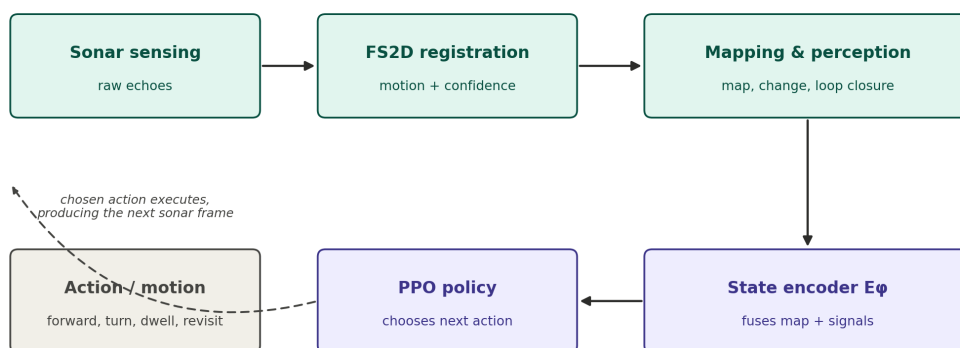
The literal equation, term by term.

Why this file exists: A pure function with no simulator state is far easier to unit-test and reason about in isolation — exactly the property that let a real reward-hacking exploit (see `tests/test_reward.py`) get caught, diagnosed, and fixed here, without touching the environment that calls it.

sim_env.py

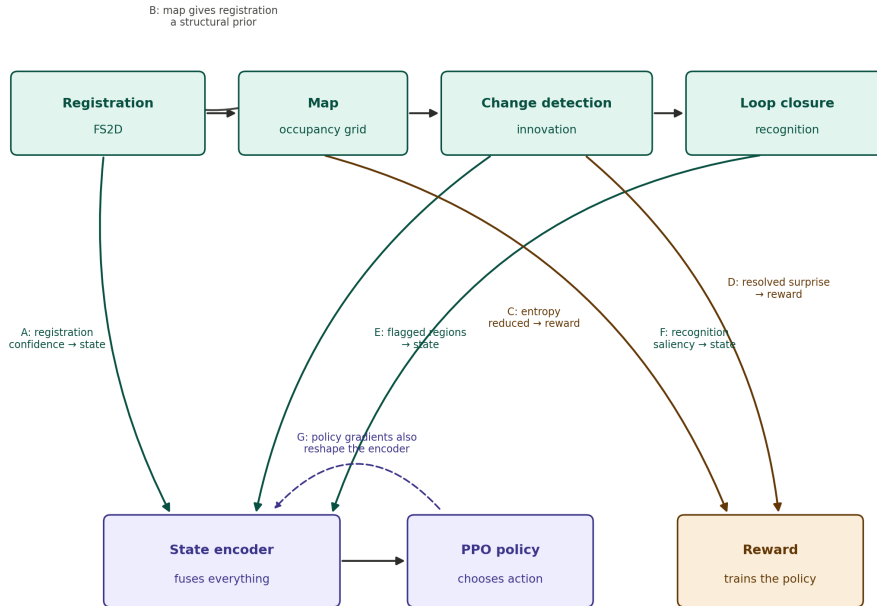
Defines `EnvConfig` and ActiveSlamEnv`: the Gymnasium environment whose step() method is the literal orchestration of every module above, in sequence, plus the true-vs-estimated pose bookkeeping that makes drift and loop-closure correction visible.`

The active-SLAM perception → decision → action loop



The full perception-decision-action loop this file orchestrates.

Seven feedback loops close the perception → decision loop



The seven feedback loops, each a specific line inside `step()`.

Why this file exists: Something has to call registration, then mapping, then change detection, then loop closure, then build the observation, in the right order every step — this file has almost no logic of its own precisely because that's not its job; its job is wiring.

`marinegym_env.py`, `stonefish_env.py`

Each defines a subclass of `ActiveSlamEnv`` (`MarineGymActiveSlamEnv``, `StonefishActiveSlamEnv``) overriding exactly three methods (`_reset_internal_state``, `_apply_action``, `close``), plus small adapter classes translating each real simulator's native interface into the same `.is_free()`/`.occ`` and `.sense_imaging()`/`.sense_scanning_360()` shape `TunnelWorld`/`SonarModel`` already provide. See `docs/MARINEGYM_INTEGRATION.md` and `docs/STONEFISH_INTEGRATION.md` for the full, verified detail on each.

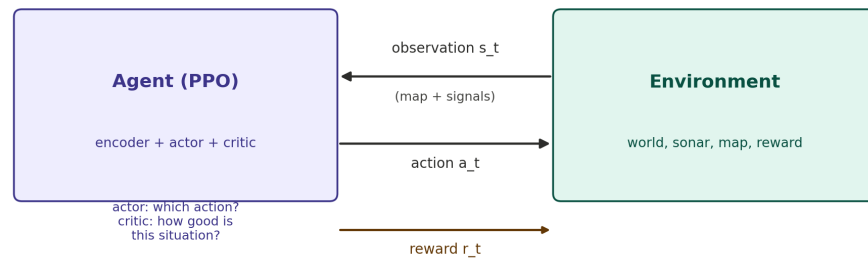
Why this file exists: Overriding only 3 methods means every other module in this entire repository — registration, mapping, change detection, loop closure, reward, encoder, PPO, metrics, plotting — keeps working completely unmodified against a real simulator. That guarantee only holds because `sim_env.py`'s` step()` never talks to the world/sonar through anything other than the four methods these adapters replace.`

src/active_slam_rl/rl/

train.py

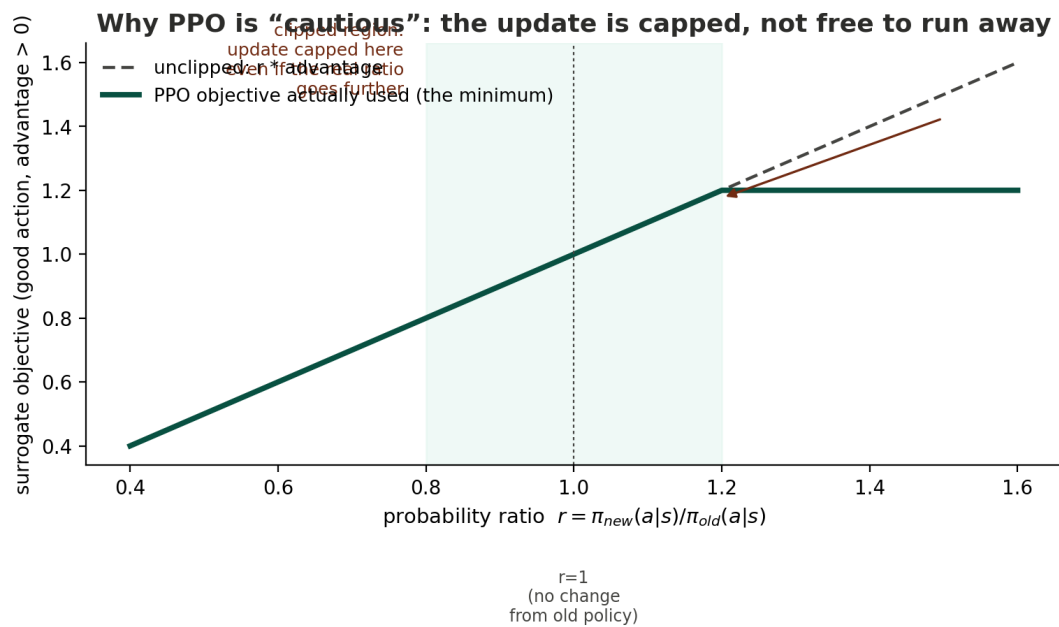
Defines `make_env()`/`build_vec_env()` (wraps `ActiveSlamEnv` for Stable-Baselines3) and `train()` (constructs PPO with `SB3StateEncoderExtractor` as its features extractor, attaches `MetricsLoggingCallback`, runs `model.learn()`).

The reinforcement-learning loop



good actions get nudged to be more likely next time — cautiously, one small step at a time

Observe, act, receive reward; actor decides, critic judges.



The literal mechanism behind PPO's caution: the smaller of the plain and clipped update.

callbacks.py

Defines `MetricsLoggingCallback`: writes one CSV row per completed episode (reward, completeness, ATE, collisions, loop closures), and — the live-monitoring feature — re-plots and overwrites `training_curves.png` every few episodes while training is still running.

baselines.py

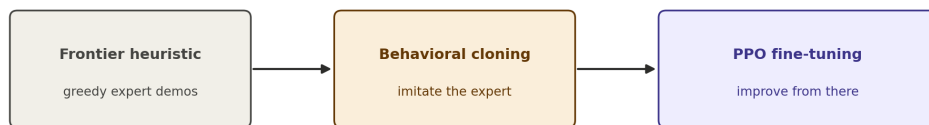
Defines `RandomWalkPolicy`, `FrontierBasedPolicy` (greedy toward unknown space), `NextBestViewPolicy` (greedy one-step uncertainty reduction) — fixed heuristics, never trained.

behavioral_cloning.py

Defines `collect_demonstrations()` (rolls out an expert heuristic, records observation/action pairs) and `behavioral_clone()` (supervises the PPO policy's log-probability of the expert's action — classification-style training on the *same* joint encoder PPO will keep fine-tuning).

Bootstrapping the policy from expert demonstrations

Stage 1: learn to imitate before learning to optimize



without this: early random exploration bumps into walls before ever stumbling onto a rewarding strategy — a slow, painful cold start

Imitate a reasonable expert before ever weighing risk against reward alone.

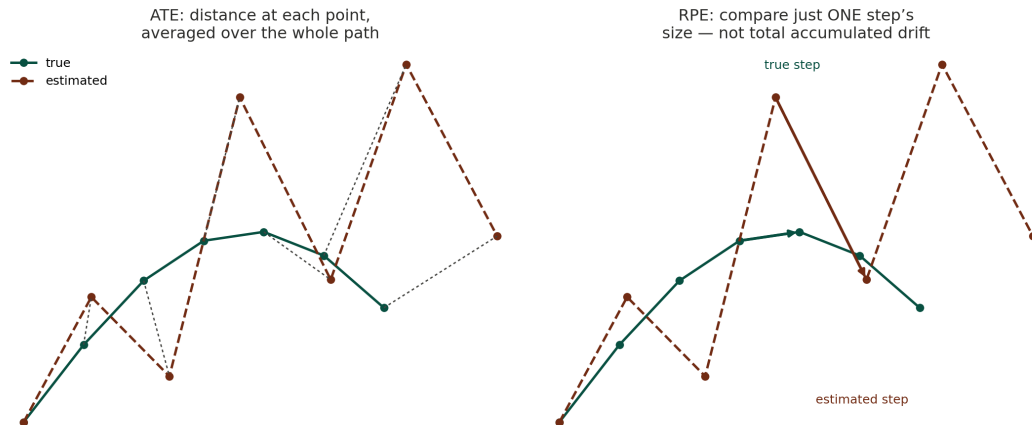
Why this file exists: These three files are split because they answer three different questions — **how do we learn** (train.py), **how do we know it's learning** (callbacks.py), and **what do we compare against, or bootstrap from** (baselines.py, behavioral_cloning.py) — and each is genuinely useful without the others (you can run baselines with zero training at all).

src/active_slam_rl/metrics/

evaluation.py

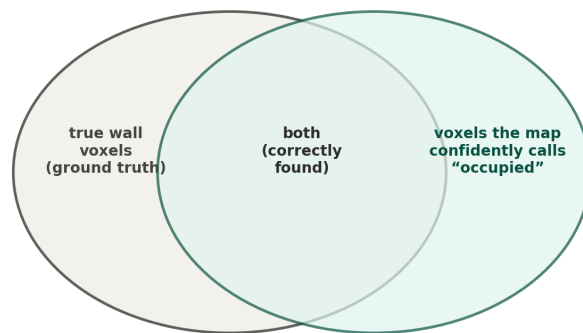
Defines `EpisodeMetrics` (the full bundle: completeness, ATE, RPE, path length, info gain/meter, collisions, loop closures, and raw traces for plotting), `relative_pose_error()`, `rollout_episode()` (runs any policy — heuristic or trained — and returns the bundle), and `summarize_runs()` (mean±std across episodes).

Two different ways to measure “how wrong is the trajectory estimate”



ATE totals drift at every point; RPE looks at just one step at a time.

Map completeness: how much of the real structure did we actually find?



$$\text{completeness} = \frac{|\text{reconstructed} \cap \text{ground truth}|}{|\text{ground truth}|} \times 100\%$$

Completeness is a strict overlap, not partial credit.

plotting.py

Defines `plot_training_curves()`, `plot_episode_dashboard()`, `plot_baseline_comparison()` — every PNG this project produces goes through one of these three functions.

Why this file exists: Separating “how do we roll out and measure a policy” (evaluation.py) from “how do we draw that as a picture” (plotting.py) means you can compute metrics inside a headless training loop without ever importing matplotlib, and reuse the exact same numbers for a live plot, a final report figure, or a raw CSV.

src/active_slam_rl/visualization/ and utils/

visualization/live_viewer.py

Defines `record_rollout_frames()` (runs an episode, keeps only the lightweight per-step state needed for animation) and `render_gif()` (draws the live occupancy belief plus true/estimated trajectories into an animated GIF).

utils/config.py

Defines `load_config()`: reads a YAML file and builds the nested `EnvConfig`/`WorldConfig`/`SonarConfig`/`RewardWeights`` dataclasses from it.

Why this file exists: Every other module takes its settings as typed dataclass arguments, never reads a config file itself — this is the one place YAML parsing happens, so the rest of the codebase never needs to know configuration comes from a file at all.

scripts/

Every file here is a thin CLI wrapper — the only files in the repo meant to be run directly rather than imported.

<code>run_training.py</code>	<code>train()</code> with a <code>--config</code> and <code>--timesteps</code> override
<code>run_eval.py</code>	<code>rollout_episode()</code> across baselines + a trained model, saves plots
<code>visualize_demo.py</code>	<code>record_rollout_frames()</code> + <code>render_gif()</code> for one policy
<code>run_bc_then_ppo.py</code>	<code>collect_demonstrations()</code> -> <code>behavioral_clone()</code> -> <code>train()</code>
<code>live_monitor.py</code>	a standalone on-screen matplotlib window polling the training CSV

Why this file exists: Keeping all argument-parsing and file-path logic out of `src/active_slam_rl/` means every importable module stays a plain, testable Python API — the scripts are the only layer that knows about command-line arguments at all.

tests/

test_registration.py	FS2D recovers known rotations/translations; quality/covariance sanit
y	
test_mapping.py	OccupancyGrid updates correctly; entropy decreases; change detection
test_env.py	Gym contract: valid observations, termination, seeded determinism
test_reward.py	regression tests locking in the reward-hacking fix
test_marinegym_adapter.py	structural checks: imports cleanly, correct overrides, no ISAAC need
ed	
test_stonefish_adapter.py	structural checks: imports cleanly, correct overrides, no ROS2 neede
d	

Why this file exists: Every file above this point in the repo is exercised by at least one of these — 26 tests total. The two adapter test files exist specifically so a future change can't silently break the MarineGym/Stonefish integration for people who don't have either installed to actually run against.

docs/

<code>active_slam_rl_explained.pdf</code>	concept-flow walkthrough with diagrams (companion to this doc)
<code>FILE_BY_FILE_GUIDE.pdf</code>	this document
<code>ARCHITECTURE.md</code>	the concept walkthrough in plain markdown
<code>MARINEGYM_INTEGRATION.md</code>	verified install steps + honest scope of what's public
<code>STONEFISH_INTEGRATION.md</code>	a stronger immediate alternative, with real sonar today

That's the whole repository. If you're looking at a file and it isn't mentioned above, it's almost certainly a generated artifact under `results/` — a trained model, a CSV log, or a PNG/GIF produced by running one of the scripts above, not something written by hand.